Music Historeum — Repository Audit: Issues & Improvements

Context

The user asked for a general review of the repository to surface issues and improvement opportunities. This is a report, not a change request — nothing below has been implemented. Three parallel read-only audits covered the Express/Postgres server + Python ingestion script, the client's Redux/services layer, and the client's UI components/pages/shared data. Findings are ordered by severity within each area. (The previously-reported hardcoded Postgres password in `bb_script/bb_scrape.py` and its rename to `bb_scrape.py` are already handled/documented in CLAUDE.md and are not repeated here.)

Server (`server/index.js`, `db.js`, `package.json`)

Critical — SQL injection

- `server/index.js:67` — `pool.query(\`SELECT get_artist_list(${startChar})\`)` interpolates `req.params.start_char` directly into the SQL string with no parameterization, escaping, or validation. Every other query in the file uses `$1` / `$2` placeholders; this one route breaks that pattern and is directly exploitable.
- `server/index.js:106,111,116,122` — `get_weekly_${chartType}_chart` / `get_range_${chartType}_chart` interpolate `chartType` into the function name. Currently guarded by the `chartType === 'Song' || 'Album'` check at line 104, so not exploitable today, but fragile — should be an allow-list lookup instead of string building so a future refactor can't reopen it.

High — silent failures leave client requests hanging

- `server/index.js:58-60, 70-73, 89-91, 133-135` — catch blocks for `/chartList`, `/artist/list/:start_char`, `/artist/:dartist/:dtype`, and `/chart/:cid/:ctype/:ctf/:cdate` only call `logger.error(...)`; none send a `res.status/json/send`. If the query throws, the client's `fetch` hangs with no response. (`/contact` at lines 153-161 does this correctly — it's the one exception.)

Medium

- `server/index.js:141-151` — `/contact` inserts `firstName`, `lastName`, `email`, `text`, `topic` raw into the outgoing HTML email body and subject line with no escaping, validation, or length caps —

allows HTML injection into the email and unvalidated subject-line content.

- `server/index.js:14` — `cors()` with no options allows any origin; fine for dev, should be locked to production domain(s) before/at launch.
- `server/db.js:5-11` — the `pg.Pool` has no `pool.on('error', ...)` handler; an idle-client backend error (e.g. DB restart) is an uncaught exception that can crash the process.
- `server/index.js:18-25` — Winston is configured with only a `File` transport; running `node index.js` locally prints nothing to the terminal.
- `server/index.js:140` — `logger.info(req.body)` on `/contact` persists submitter name/email/message in plaintext to `mh_server.log` indefinitely, with no redaction.
- `server/index.js:66,165` — multi-argument `logger.info('label', value)` calls rely on splat-style interpolation, but the format chain (`combine(timestamp(), json())`) has no `splat()` — the second argument is likely dropped from the rendered log line.

Low

- `server/package.json` — no `"start"` script (must run `node index.js` directly), no dev/nodemon script, no `engines` field. Dependency versions (`express ^4.18.2`, `nodemailer ^6.9.1`, `pg ^8.8.0`, `winston ^3.8.2`, `dotenv ^16.0.3`) are all a couple of years behind current — worth an `npm outdated` pass.
- Root `.gitignore` only ignores the literal path `server/mh_server.log`, not a `*.log` wildcard — doesn't cover `bb_script/billboard.log` / `billboard.txt` (both untracked-directory output files) or any renamed log.

Python ingestion script (`bb_script/bb_scrape.py`)

Medium — correctness

- Lines 294-295 — when a chart date's entries are all duplicates (`entries_entered == 0`), the code sets `last_date = True` but never calls `updateChartList()` (unlike the two branches above it). Since `getChartDate()` derives the next fetch date from `chart_list.next_date` / `last_date` in the DB, that chart's cursor never advances — future runs re-fetch and re-skip the same date forever.
- No reconnect/retry logic: a single `psycopg2` connection (line 11) is opened once and reused for the entire run (potentially thousands of iterations back to `default_date = '1950-01-01'`, with a `time.sleep(10)` between each). Only `insertChartEntry` narrowly catches `psycopg2.Error` (for duplicate-key `23505`); any other dropped connection or DB error kills the whole run and loses all in-memory progress.
- No error handling around the Billboard scrape calls themselves (`billboard.charts()`, `billboard.ChartData(...)`) — a scrape/HTTP failure is an uncaught exception. The only related

exception handling (`except AttributeError`, lines 296-299) guards `chart.nextDate` access, not the scrape call.

Low — dead code / cleanliness

- `active_song_lists`, `active_albums_lists`, `active_artists_lists`, `active_greatest_lists` (lines 14-17) are declared but never populated (their only writer is commented out at lines 74-82).
- `testEntries()` (lines 45-61) references `chart_name` / `chart_date` that don't exist in its own scope — broken, unreachable, has no call site.
- `time.sleep(10)` repeated 3x as a magic number with no named constant/comment.
- No docstrings/type hints; redundant `print(...)` + `logging...(...)` pairs throughout instead of relying on the logger alone.

Client: Redux/services layer

High — no error handling anywhere in the fetch layer

- `services/fetchChartList.js`, `fetchChartData.js`, `fetchArtistData.js`, `fetchArtistList.js` — none check `response.ok`; all call `.json()` unconditionally. A non-2xx response with an HTML error body throws inside `.json()`; a network failure makes `fetch` itself reject. Nothing catches either case anywhere upstream (not in the services, not in `App.js`, not in page components).
- `services/fetchContactForm.js:11-19` — no `.catch`, and the `.then` callback returns `undefined`, so the function always resolves to `undefined` regardless of success or failure; callers cannot detect failure at all.
- **Compounding UX bug**: `App.js:25,51` — the entire app (`Header` / `Routes` / `Footer`) only renders when `dataLoaded` is `true`, with no loading spinner or error fallback. If the chart-list fetch fails or is slow, the user sees a permanently blank page. Additionally, `setDataLoaded(true)` (line 35) is called *inside* the `chartList.forEach` loop — if the API ever returns an empty array, `dataLoaded` never becomes `true` either.

Medium

- `counter` feature (`Counter.js`, `counterSlice.js`, `counterAPI.js`, `counterSlice.spec.js`) is unmodified Redux Toolkit template boilerplate — confirmed not imported by `store.js` or any real component. Dead code, safe to delete.
- `App.test.js` is the stock CRA "learn react" test, asserting text that doesn't exist in this app; it's also missing a `BrowserRouter` wrapper around `<App/>`, so it doesn't meaningfully exercise anything and would fail. Should be rewritten or removed.
- `baseUrl.js` confirmed hardcoded (`http://localhost:5000/`, prod URL commented above). Grep across `client/src` for `process.env.REACT_APP` returns zero matches — no env-based config exists

anywhere in the client, making this a clean, isolated fix (`REACT_APP_API_BASE_URL`).

- `features/chartMenu/chartsMenusSlice.js` (`updateLastDate`) — shallow-copies `state.songChartsArray` into `newChart`, mutates `newChart[newIndex]` (the *same object reference* as the original), but never reassigns `newChart` back to state. It "works" only because Immer intercepts mutations anywhere in the draft tree; the copy itself is dead code. Should just mutate `state.songChartsArray[newIndex].LastDate` directly.
- `chartsSlice.js` (`updatePendingType` / `updatePendingTimeframe`) — hardcoded magic-string comparisons (`'1'`, `'2'`, `'3'`) against `action.payload`, presumably a `<select>` value, with no validation or default/warning if the value doesn't match.

Low

- ~54 `console.log` calls across 17 client files; several dump full Redux state or raw API payloads on every dispatch/render (e.g. `artistsSlice.js`'s `getTopArtists()` selector logs on every call since it's invoked inline in `useSelector`).
- `package.json` dependency bloat: three overlapping styling systems (`bootstrap` + `reactstrap` + `styled-components`); `font-awesome` v4 alongside Bootstrap's own iconography; five separate font packages; `redux-logger` declared but never wired into `store.js`; `disconnect` appears unused anywhere in `src`; `react-table` v7 (pre-hooks API) — worth confirming it's still the right fit.

Client: UI components, pages, utils, shared data

High — silent failures / broken error paths

- `features/contact/ContactForm.js:22-23` — calls `fetchContactForm(comment)` without `await` / `.catch` and closes the modal unconditionally regardless of success. Combined with the `fetchContactForm.js` issue above, a failed submission is completely silent — the user believes their message sent even if it didn't, with no success/failure feedback anywhere.
- `features/artist/ArtistCard.js` (`fetchData()` in `useEffect`) — no try/catch around its two `fetchArtistData` calls; a thrown error (network failure, or the TypeError below) becomes an unhandled rejection and the card silently never renders.
- `services/fetchArtistData.js` — no `response.ok` check; `data[0].get_songs_by_artist` throws a `TypeError` if the API returns an empty array or error payload (compounds directly into the ArtistCard issue above).

Medium — accessibility

- `components/Table.js:237` — the only way to navigate from a chart row to an artist page is `<td onClick={...}>` with no `role="button"`, `tabIndex`, or keyboard handler — completely inaccessible to keyboard-only users.

- `features/contact/ContactForm.js:29` — `<a href style={...} onClick={...}>` with no actual `href` value; browsers drop such anchors from the focus/accessibility tree. Should be a `<button>`.

Medium — stale/hardcoded data

- `app/shared/SONG_CHARTS.js` and `ALBUM_CHARTS.js` — every chart's `LastDate` is hardcoded to `"2023-02-18"`, now (2026-07-13) over 3 years stale; should come from the API rather than being baked into the bundle.
- `SONG_CHARTS.js` — `FirstDate` force-set to `"1962-01-01"` for several charts with the true historical start dates left commented out rather than removed (undocumented workaround + dead code).
- `app/shared/TOP_ARTISTS.js` — fully hand-maintained static list of ~44 artists with hardcoded sales figures; contains a commented-out AC/DC entry and a typo (`'Barbra Steisand'` → "Streisand"). Should be DB-driven like everything else, or at least cleaned up if it stays static.

Low — duplication / cleanliness

- `utils/WeekPicker.js`, `MonthPicker.js`, `YearPicker.js`, `DecadePicker.js` each duplicate an identical styled-component block and an identical, unexplained `onChange` hack (`document.getElementById('root').dispatchEvent(new MouseEvent('click', {shiftKey: true}))`) — good candidate for extraction into one shared component/hook. Same files also carry leftover copy-paste console logs still labeled `'WeekPicker Before/After'` in the other three pickers.
- `features/artist/ArtistStyles.js` and `features/chart/ChartStyles.js` are near-byte-identical styled-components — should be a single shared component.
- `features/chart/ChartNav.js` — a whole commented-out duplicate `useLayoutEffect` block sits next to the live one.
- `components/SubHeader.js` — a commented-out `<h2>` left in place.
- Numerous leftover debug `console.log`s scattered through `ArtistCard.js`, `ChartCard.js`, `ChartNav.js`, `AlphabetNav.js`, and the date-picker utils.

Suggested priority if you want to act on this

1. **Fix the SQL injection** at `server/index.js:67` (parameterize `get_artist_list`).
2. **Add error responses** to the four silent server catch blocks, and **error/loading handling** in `App.js` + the client `services/*` fetch wrappers — these two gaps compound into "blank page forever" and "silent contact-form failure," the most user-visible problems in the app.
3. Sanitize/validate the `/contact` route inputs.
4. Clean up dead code (`counter` feature, unused Python vars/function, commented-out blocks) and stale data (`SONG_CHARTS` / `ALBUM_CHARTS` `LastDate`, `TOP_ARTISTS` typo).

5. Everything else (dependency pruning, console.log cleanup, accessibility, DB pool error handler, CORS lockdown) as time allows.

Generated 2026-07-13 — Music Historeum repository audit.